

Tarea Grupal

Arquitectura de Datos, BI y ML en GCP para un Banco

Objetivo General

Diseñar una arquitectura end-to-end en GCP (datos, BI y ML) que resuelva necesidades típicas de un banco: detección de fraude en tiempo casi real y analítica de clientes (KPIs y segmentación). La solución debe contemplar ingesta, procesamiento, gobierno de datos, almacenamiento, analítica, inferencia de ML y visualización ejecutiva.

Trabajo en Equipo:

- Grupos de 2–4 integrantes.
- Todos deben figurar en portada y README.
- La arquitectura debe ser clara, justificable y escalable.

Etapas del Proyecto:

1) Definición del Problema (Banco)

- Caso:
 - Fraude: identificar transacciones sospechosas en ~tiempo real para alertar/retener.
 - BI de Clientes: tableros con KPIs (Nº transacciones, monto, tasa de rechazo, alertas de fraude, segmentos).
- Fuentes de datos (ficticias/ejemplo):
 - Transacciones de tarjetas (streaming).
 - Maestro de clientes y productos (batch).
 - Catálogo de comercios (batch).
- Requisitos no funcionales: latencia baja para scoring de fraude, seguridad por dominios (principio de mínimo privilegio), auditoría, costos controlados, alta disponibilidad.

Entregable: Resumen (≤1 página) con objetivos, fuentes, KPIs y supuestos.

2) Diseño de Arquitectura (Draw.io / diagrams.net)

Incluye (con íconos GCP y flechas etiquetadas):

- Ingesta
 - *Streaming*: Pub/Sub recibe eventos de transacciones.
 - *Batch*: archivos diarios en Cloud Storage (clientes, comercios).
- Procesamiento
 - *Stream*: Dataflow (Apache Beam) para limpieza, enriquecimiento (join con features en BigQuery/Redis opcional) y envío a destinos.
 - *Batch*: Cloud Composer (Airflow) orquesta cargas hacia BigQuery.
- Almacenamiento/Analytics
 - BigQuery como *warehouse* (zonas: *raw*, *staging*, *prod*).
 - BigQuery BI Engine para acelerar dashboards.
 - Looker Studio (o Looker) para tableros de fraude y KPIs.

- ML
 - *Entrenamiento*: Vertex AI (Workbench/AutoML) usando datasets de BigQuery.
 - *Registro de modelos*: Vertex AI Model Registry.
 - *Despliegue*: Vertex AI Endpoints (online) para scoring con baja latencia o batch scoring con Dataflow/BigQuery ML.
 - *Features*: (opcional) Vertex AI Feature Store para consistencia entre entrenamiento/serving.
- Serving / Integración
 - Cloud Run expone un microservicio (sin código en esta tarea) que orquesta solicitudes al endpoint de Vertex AI y publica alertas (vía Pub/Sub) o escribe resultados en BigQuery.
- Gobierno, Seguridad y Operación
 - Cloud IAM (roles mínimos), VPC Service Controls (si aplica), Audit Logs.
 - Data Catalog (metadatos), Cloud KMS (claves).
 - Cloud Monitoring / Logging para métricas y trazas (latencia, errores, throughput).
 - Budgets & Alerts para control de costos.

Entregables:

- Archivo .drawio y exportación .png o .pdf.
- El diagrama debe tener capas/zonas (Raw → Curated/Prod), nombres de tablas clave y puntos de control (validaciones, DLQ si aplica).

3) Justificación Técnica

- Explicar por qué cada servicio GCP: latencia, elasticidad, costo, facilidad de operación.
- Describir flujo de datos (stream y batch), líneas de defensa (calidad/validación), seguridad (IAM, KMS), y operabilidad (logging/monitoring).
- Mencionar riesgos y mitigaciones (p.ej., picos de tráfico, esquemas cambiantes, *data drift*).

Entregable: Documento (1–2 páginas) con la justificación y el diagrama referenciado.

4) Publicación en GitHub

Repositorio público: arquitectura-gcp-banco-[grupo] con:

- Carpeta /diagramas (.drawio, .png/.pdf).
- /docs con Definición del problema y Justificación técnica.
- README.md con:
 - Descripción del caso, diagrama embebido, KPIs, decisiones técnicas.
 - Guía de cómo leer el diagrama y supuestos.
 - Roles del equipo.

Entregable: Enlace al repositorio.

5) (Plus opcional) Extensiones

- Costeo aproximado (alto nivel) por componente (estimación mensual).
- Estrategia de calidad de datos: reglas, monitoreo de SLA, DLQ.
- MLOps: ciclo de retraining, evaluación (AUC/PR, *drift*), *canary* o *shadow deployment* en Vertex AI.

Entregable: Anexo en /docs (1 página máx.).



Requerimientos de Contenido Mínimo del Diagrama

- Trazado end-to-end (ingesta → procesamiento → almacenamiento → ML → BI).
- Servicios GCP rotulados y flujos con latencia esperada (batch vs. stream).
- Tablas/datasets en BigQuery (ej.: raw.transactions, curated.features, bi.kpis).
- Punto de inferencia online (Vertex AI Endpoint) y salida a alertas/tabla.
- Controles: IAM, KMS, Logging/Monitoring y dominio de costos.



KPIs sugeridos (BI)

- Tasa de fraude sospechado vs. confirmado.
- Latencia promedio de scoring (P95).
- Volumen transaccional por hora/día.
- Pérdida evitada estimada.
- Tasa de falsos positivos (si se modela).



Entrega

- Modalidad: Grupal (2–4).
- Fecha límite: domingo – 19/10/2025
- Formato:
 - .drawio + .png/.pdf del diagrama.
 - Documento de problema y justificación (PDF, 1–2 páginas).
 - Enlace al GitHub.



Recomendaciones

- Usen plantillas/íconos oficiales de GCP en Draw.io.
- Mantengan simplicidad y narrativa (menos es más; etiqueten flujos).
- Alineen diagrama con KPIs; indiquen dónde se calculan.
- Señalen controles de seguridad y observabilidad (no solo “cajas”).
- En README incluyan una vista lógica (qué resuelve) y una vista física (servicios).

**Criterios de Evaluación (Nota máxima: 7.0)**

Criterio	Descripción	Ponderación	Puntaje Máximo
1. Definición del problema	Claridad del caso bancario, objetivos y KPIs.	15%	1.05
2. Diagrama en Draw.io (GCP)	Compleitud, legibilidad, conexiones correctas, zonas de datos y flujos batch/stream.	35%	2.45
3. Justificación técnica	Elección y trade-offs de servicios (latencia, costos, seguridad, operación).	25%	1.75
4. Publicación en GitHub	Estructura, README claro con diagrama embebido y documentación organizada.	15%	1.05
5. Trabajo en equipo	Coherencia visual, ortografía, roles y consistencia general.	10%	0.70
Plus (opcional)	Costeo alto nivel, estrategia de calidad de datos o ML.	+0.3	+0.30
Total máximo posible		100% + bonus	7.x